Challenges in real-time-embedded IoT Command Recognition

Hazem Younis, John H.L. Hansen
CRSS-CI Lab: Center for Robust Speech Systems – Cochlear Implant Processing Lab
Department of Electrical & Computer Engineering, The University of Texas at Dallas, Richardson, USA

***Abstract*— Automatic speech recognition (ASR) is an effective means of communicating information for command and control of IoT based electronic devices. Voice enabled electronic devices offer the prospect of greater user control and increased system support for deployed platforms. The internet of things (IoT) is an internet-based architecture that allows for data transmission over wireless networks for control and monitoring. By combining previously stated architectures, we can perform many daily tasks remotely, eliminating the need for physical presence. Some challenges associated with command recognition include command word perplexity as well as noise levels/distortion in the environmental setting. In this study, system formulation consisted of (i) a Raspberry-pi low resource microprocessor with fixed computing capacity, and (ii) a ReSpeaker microphone system, which provided an integrated hardware setup for algorithm development to combine aspects of ASR and IoT, allowing users to dictate a variety of commands remotely. The platform eliminates the need for physical interaction with the device by simply listening to commands and performing actions to fulfill the command. The platform is suitable for several settings including residential and commercial spaces due to its low computational resource model. Trade-offs in machine learning based CNN network design and training are explored. Also, challenges and factors that affect speech-command recognition for IoT related applications are considered. A summary of phonetic and computational limiting factors for success in command recognition devices also discussed. The study highlights several ASR based neural net architectures which are integrated within a low-resource computing platform, along with benchmark performance evaluations.**

Keywords – automatic speech recognition, low-resource computing, IoT, command recognition.

I. INTRODUCTION

Automatic speech recognition (ASR) is generally viewed as the most probable word or word sequence that describes a given audio. Numerous ASR systems have been developed for various applications throughout a diversity of domains [10]. There are several factors that affect the performance of an ASR system. In addition to the computational capabilities of the system, the quality of audio data used to train the system, as well as speaker variability/mismatch can greatly impact the system's accuracy[11]. To understand the speech utterances and how they are recognized by a computer, we need to understand the underlying linguistic theory [5]. Speech is a combined sequence of modulated sounds that form utterances. These "distinctive sound units" are known as phonemes. The phonetic structure of a word significantly affects the word's pronunciation variability. Table 1 illustrates the NATO phonetic alphabets which were chosen specifically among the English language for their distinct names that are easily identified among remote speech communicators. NATO has also address the impact of speech under stress in naturalistic settings on military speech technology, as well as ways to mitigate such mismatch[12].

The field of ASR not only evolved tremendously, but also integrated other technological domains such as IoT. A specific application that incorporates aspects of both domains is speech-command recognition. Command recognition is one of the simplest forms of ASR. That is mostly due to the lack of complexity in contrast to recognizing a full sentence or paragraph. There are several spaces where command recognition is impactful. These domains include, but are not limited to, emergency detection [2], smart home integration, & commercial production. A challenging aspect of command recognition is the flexibility required of the system. The device must be portable enough to integrate among several environments. For the device to meet these requirements there are limits on its computational capacity. Fortunately, command recognition typically involves detecting a combination off one to three utterances that are classified as a command. In this, a high-level overview of the factors affecting the quality of command recognition on an embedded system is provided in addition to the results acquired from experimenting with various algorithms and computational platforms.

TABLE 1: NATO PHONETIC ALPHABET

A	Alpha	N	November
B	Bravo	O	Oscar
C	Charlie	P	Papa
D	Delta	Q	Quebec
E	Echo	R	Romeo
F	Foxtrot	S	Sierra
G	Golf	T	Tango
H	Hotel	U	Uniform
I	India	V	Victor
J	Juliet	W	Whiskey
K	Kilo	X	X-ray
L	Lima	Y	Yankee
M	Mike	Z	Zulu

II. COMPUTATIONAL PLATFORMS

A. *Beaglebone Black*

The Beaglebone Black (BBB), demonstrated in Fig.1, is one of the several ARM microprocessors developed by Texas Instruments for various IoT applications. Beaglebone black is regarded as the most utilized Beagle bone board amongst all the boards available till date for various applications and hardware-based projects and has support of developers and hobbyists' communities worldwide. It is powered by AM335x 1GHz ARM Cortex A8 processor and supports 3D graphics acceleration with a 512MB RAM capacity [1]. While the Beaglebone is an attractive platform in terms of portability and computational capacity, the disadvantage it presents is its configurations. Although the device is superior to those that are akin to it in terms of computational capacity, the BBB requires a specialist to understand its hardware on a kernel level to be able to modify the device to accept different audio inputs and outputs. For the purposes of this research experiment, the Raspberry-pi illustrated in Fig.2 was used with a Respeaker microphone attached.

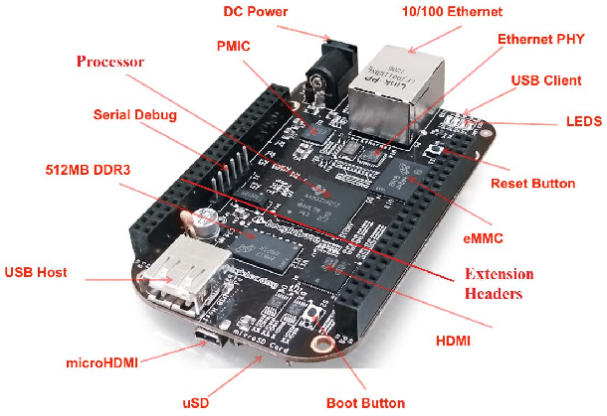


Fig. 1. Beaglebone Board Components [Img Src: www.beagleboard.org]

B. *Raspberry-Pi with Respeaker Microphone*

The Raspberry-pi is a computing platform that is very popular among academic institutions due to its feasibility and ease of use. The raspberry pi is approximately the size of a credit card and is comparable to the Beaglebone. The pi used for this project's purpose is the "Raspberry-pi 3 model B+". This version of the platform functions at 1.4GHz Cortex-A53 (ARMv8) with a 1 GB RAM capacity. The Respeaker microphone is a separate unit that is designed to easily plug into the raspberry pi through the designated I/O connector pins. The Respeaker version used includes a 4-mic array in addition to 12 APA102 programmable LED's for visual aid. When compared with the Beaglebone, the raspberry pi was found to be more powerful & faster in terms of computational capacity and has more storage memory. In addition, the raspberry-pi's feasibility suggested its ease of integration for this study's purpose.

III. DATA PORTFOLIO

The dataset used for this study is the Google Speech Commands Dataset [7]. The dataset is a crowd-sourced collection of 65,000 utterances of 36 short words including commands such as "go",

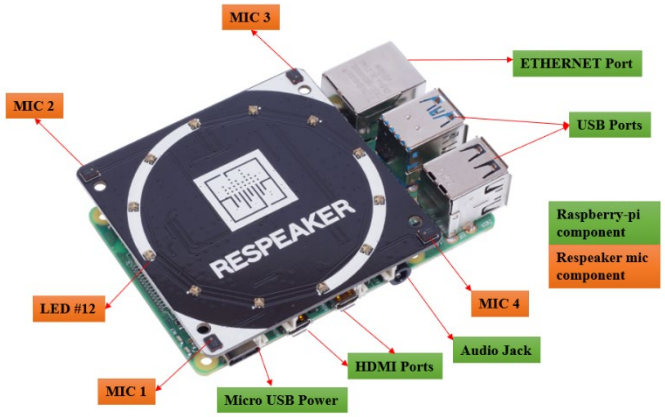


Fig. 2. Raspberry-pi & Respeaker Microphone components

"follow", "off", & "stop". The dataset was gathered by prompting volunteers to speak single-word for one second duration. The advantage to this dataset which makes the training process significantly simpler is the consistency in utterance lengths. The training process is discussed in a later section in this paper. The dataset also includes over 20,000 samples from 10 non-command words. The purpose of these non-command utterances is to give the dataset some variability and test the performance of the developed command recognition system. While this dataset is feasible and maintains low pre-processing requirements, the metadata for this dataset is not released due to the dataset being open source and compliant with speaker privacy protection. For the purposes of this study, the interest was to examine a small-footprint command recognition system for an embedded system. Therefore, only five commands were selected: "follow", "stop", "go", "learn", & "off". Each speaker is labeled with a unique ID through the outsourced dataset collection algorithm. However, since information about the speakers are missing, we were able to at least evaluate the effect of each member's contribution to the dataset on the performance of the command recognition system. Each of the selected commands contained several utterances ranging between 1500 & 4000 1-second audio clips. 1300 pseudo-random clips were selected among each word and used as training data. Figures 3 to 5 describe the contribution or token density of people who provided a certain number of samples.

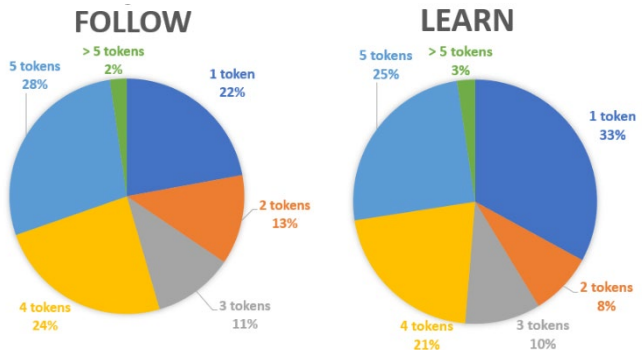


Fig. 3. Speaker Token Density for "Follow" & "Learn" Commands



Fig. 4. Speaker Token Density for "Go" & "Stop" Commands

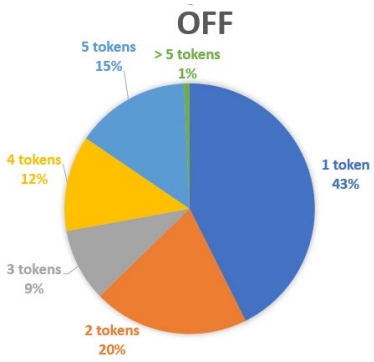


Fig. 5. Speaker Token Density for "Off" Command

IV. Algorithm & System Development

Two similar but distinct algorithms were developed to train the five command models separately. The algorithms were developed in a Python IDE known as "Spyder" using TensorFlow and Keras libraries. A simple binary classifier as used to determine whether the utterance spoken matched the modeled command. The pipeline for both the algorithms developed followed the following structure: Data Collection, Feature Extraction, Model Training, & Deployment. The first algorithm made use of a traditional Convolutional Neural Network (CNN). The data collected was assigned labels, each label corresponding to the word resembled by the utterance. For example, all audio clips containing the word "off" were labeled "1", clips containing "go" were labeled "2", etc. The labels are later used for classification purposes. Feature extraction was then performed by acquiring the Mel-Frequency Cepstral Coefficients (MFCC). To calculate the MFFCs of each audio file, a Fast Fourier Transform (FFT) was performed on several samples of each clip. The samples were set to 250 millisecond frames with a 0.05 second offset. The FFT results in the Power of the speech signal corresponding to the signals frequencies. Filtering operations are applied to the result of the FFT. The sum of powers in each filter is then extracted and 26 Energy values corresponding to each filter are assigned to an array. The mathematical log(x) of these energies is computed since the filters are logarithmically spaced above 1kHz frequencies. Finally, a Discrete Cosine Transform (DCT) which de-correlates the values acquired and extracts real values from the signal. The resulting values from the DCT operation are referred to as MFCCs which are the features that describe the characteristics of the audio signal. This process is repeated for each sample of every audio file and results in a matrix of MFCCs for each audio clip. The result is a 16x16 matrix of MFCCs with each matrix corresponding to one audio file. It is important to note that data samples typically exhibit non-idealities such as an incomplete word or phrase recording due to human error in timing the "record" or "stop" button. To overcome this issue of problematic data, the original dataset was processed to identify data that did not contain enough information for extraction, i.e., the clips were not a full second in length. The problematic data was discarded for the purposes of this study. The extracted features were saved in a .npz file used for training leach model. Training was performed in a round robin fashion for each model with a 10% validation & 10% test set. The average of the test set accuracies was acquired and reported in Table II. The training process involved using two distinct but similar neural net architectures. The architectures can be summarized as follows:

A. Convolutional Neural Network (CNN)

Neural networks have become the preferred method of developing machine learning algorithms among many researchers. It is not unusual to integrate one or several neural nets in an intelligence or IoT domain due to the characteristics of neural nets and their accuracy in making inferences based on extracted features. The convolutional neural network composed of three convolutional layers; each layer is followed by a non-linearity "Relu" activation layer and finally a max-pooling layer. The outputs of the CNN were fed into a classifier neural net with a sigmoid activation unit used for binary classification. The output of the final layer is a prediction of whether the input MFCC matrix matches the targeted command for recognition. The overall network is trained under binary cross-entropy criterion, maximized using RMSprop algorithm described in [6].

B. Depth Seaprable Convolutional Neural Network (DSCNN)

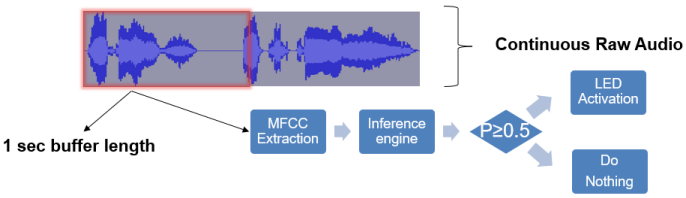
The second architecture developed for this experiment is very similar to the first neural net. In this algorithm, we explore a variation of the traditional CNN, the depth separable convolutional network. In this architecture, the advantage is that the model parameters are significantly less than a traditional CNN [8]. This property of a DSCNN is unique because it allows for ease of scalability as exhibited in [8]. The structure of this neural net is like the previously mentioned architecture with the substitution of separable convolutional layers for regular convolutional layers. A comparison of the model's accuracies is illustrated later in this paper.

V. Deployment & User Engagement

The developed command models were deployed individually to the Raspberry-pi with Resepaker platform. The TensorFlow models created are too large to deploy on a low resource computing platform such as the Raspberry-pi. To overcome this challenge the TF models were converted to TFLite models which allow us to access data from the original model serially in a memory efficient and rapid manner. TFLite models are optimized for low-resource computing platforms [9].

The pipeline for model deployment is as follows: Respeaker captures raw audio data continuously, a 1-second buffer is used to store continuous frames of raw audio and MFCCs are extracted from the buffer, the MFCCs are passed to the TFLite model and an inference engine is used to determine whether the captured audio is probabilistically similar to the target command based on a user defined threshold. Since the design of this system integrates each command in a separate model, the threshold was set to 0.5 or 50%. Finally, a prediction is made by the inference engine based on the extracted MFCC's that had been compared to the developed model. If the prediction determines the command was uttered, the Respeaker is programmed to turn on its LED's as a representation of a response to the command. Otherwise, the buffer is updated, and the process is repeated. Figure 6 illustrates the described pipeline.

Fig. 6. Pipeline for system deployment on the computing platform



VI. Results

Overall, both developed systems performed well. The result included 10 developed models with algorithms used for each command. The accuracy metric was used to measure each system's performance. A comparison of system performances is summarized in Table II.

TABLE I. Model Performance Comparison Table

Model/Command	Training Algorithm	Evaluated Accuracy (%)
STOP	CNN	94.63%
	DSCNN	98.34%
LEARN	CNN	96.13%
	DSCNN	94.43%
GO	CNN	92.89%
	DSCNN	98.8%
OFF	CNN	97.3%
	DSCNN	97.87%
FOLLOW	CNN	93.75%
	DSCNN	93.67%

The evaluated results support the conclusion that both architectures performed very well. In fact, there is not a superior architecture based on the experimental values acquired. Commands such as "Go" & "Off" demonstrated very high performance within the DSCNN architecture. The simplified computations prevented over analysis of a two-phoneme commands when using the DSCNN while less-desirable results were obtained with the CNN architecture for the same commands. Commands with a higher phonetic perplexity such as "Follow" and "Learn" generally did not perform as well as shorter models. While it could be argued that the shorter commands performed better due to overfitting, it is important to note that more than 40% of the speakers in that category contributed only 1 utterance. The variability in the dataset supports the conclusion that the model was not overfit.

VII. Conclusion

This study has presented a comparison between two neural net architectures used to deploy a command recognition system on a low-resource computing platform. Both architectures were considered, and their performance metrics were examined. Several computational platforms were examined namely the Raspberry-pi and the Beaglebone Black microprocessor. The pipeline for deploying the developed recognition system on an embedded platform. The results demonstrated the success and reliability of both developed architectures. This system can be used in commercial or residential purposes in a "Smart" environment.

VIII. Acknowledgment

This study was supported by a grant from Texas Instruments, Dallas, TX. We would also like to acknowledge the Center for Robust Speech Systems at the University of Texas at Dallas for the support and resources allocated for this study.

References

[1] G. E A. Nayyar and V. Puri, "A Review of Beaglebone Smart Board's-A Linux/Android Powered Low Cost Development Platform Based on ARM Technology," *Inter. Conf. on Future Generation Communication and Networking (FGCN)*, Jeju, Korea (South), pp. 55-63, 2015

[2] E. Principi, S. Squartini, R. Bonfigli, G. Ferroni, F. Piazza, "An integrated system for voice command recognition and emergency detection based on audio signals," Expert Systems with Applications, vol. 42, 2015

[3] J. Mayer, "Low Cost Automatic Speech Recognition IOT Architecture for Hands Free Assistance to People with Mobility Restrictions," Inter. Conf. on Internet Computing (ICOMP), pp. 53-58, 2018.

[4] B. McMahan and D. Rao, "Listening to the World Improves Speech Command Recognition", AAAI, vol. 32, no. 1, Apr. 2018.

[5] J. Allen, "Linguistic aspects of speech synthesis." Proceedings of the National Academy of Sciences 92.22 (1995): 9946-9952.

[6] J. Desai, S. Bookseller, R. Mohod, C. N. Paunwala and B. Vaidya, "Performance of Different Optimizers for Traffic Sign Classification," *Inter. Conf. Computing, Communication and Networking Technologies (ICCCNT)*, Kanpur, India, 2019, pp. 1-7

[7] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition." arXiv preprint arXiv:1804.03209 (2018).

[8] Guo, Yunhui, et al. "Depthwise convolution is all you need for learning multiple visual domains." Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 33. No. 01. 2019

[9] J. Lee, et al. "On-Device Augmented Reality with Mobile GPUs.".

[10] J. Deller, J.H.L. Hansen, and J. Proakis, Discrete-Time Processing of Speech Signals, (2nd Edition), IEEE Press, New York, NY, 2000.

[11] J.H.L. Hansen, H. Boril, "On the issues of intra-speaker variability and realism in speech, speaker, and language recognition tasks," Speech Communication, vol. 101, pp. 94-108, July 2018.

[12] J.H.L. Hansen, C. Swail, A.J. South, R.K. Moore, H. Steeneken, E.J. Cupples, T. Anderson, C.R.A. Vloeberghs, I. Trancoso, P. Verlinde, "The Impact of Speech Under `Stress' on Military Speech Technology," published by NATO Research & Technology Organization RTO-TR-10, AC/323(IST)TP/5 IST/TG-01, March 2000